



(/rol/app/)

Menu

Korean

홈 (/rol/app/) (home) > Observing OpenShift Service Mesh

> Chapter 1: Observing OpenShift Service Mesh

Observing OpenShift Service Mesh

Lesson

Lab Environment

Chapter 1. Observing OpenShift Service Mesh

Introduction to Red Hat OpenShift observability and OpenShift Service Mesh

(/rol/app/courses/ad0007l-3.1/pages/ch01)

Guided Exercise: Introduction to Red Hat OpenShift observability and OpenShift Service

Mesh (/rol/app/courses/ad0007l-3.1/pages/ch01s02)

Collecting Service Metrics (/rol/app/courses/ad0007l-3.1/pages/ch01s03)

Guided Exercise: Collecting Service Metrics (/rol/app/courses/ad0007l-3.1/pages/ch01s04)

Tracing Services With Kiali, Tempo and OpenTelemetry (/rol/app/courses/ad0007l-

3.1/pages/ch01s05)

Guided Exercise: Tracing Services With Kiali, Tempo and OpenTelemetry

(/rol/app/courses/ad0007l-3.1/pages/ch01s06)

Lab: Observing OpenShift Service Mesh (/rol/app/courses/ad0007l-3.1/pages/ch01s07)

Summary (/rol/app/courses/ad0007l-3.1/pages/ch01s08)

Abstract

Goal	Trace and visualize an OpenShift Service Mesh with Red Hat OpenShift observability
Sections	• Introduction to Red Hat OpenShift observability and OpenShift Service Mesh (and Guided Exercise) • Collecting Service Metrics (and Guided Exercise) • Tracing Services With Kiali, Tempo and OpenTelemetry (and Guided Exercise)

Introduction to Red Hat OpenShift observability and OpenShift Service Mesh

Objectives

- Identify basic concepts about observability with focus in monitoring and distributed tracing.
- Identify components and key features of OpenShift Service Mesh for observability.
- Compare OpenShift Service Mesh console plug-in and Kiali console capabilities for service mesh observability.

Red Hat OpenShift Observability and OpenShift Service Mesh

Observability is the ability to understand a system's state by collecting and analyzing its output and logs. Red Hat OpenShift Service Mesh runs on Red Hat OpenShift, which always includes Red Hat OpenShift default tools for observability.

Red Hat OpenShift observability collects logs, traces, events, and system metrics to provide real-time monitoring.

Key features of the Red Hat OpenShift observability portfolio are as follows:

OpenShift Logging

Red Hat OpenShift Logging aggregates all the logs from the pods and nodes of an OpenShift cluster to a centralized location. Centralized logging improves the searching, visualizing, and reporting of data.

Network Observability

Network observability monitors and analyzes network traffic, and helps to resolve connectivity issues.

Two OpenShift observability features are intensively used by OpenShift Service Mesh:

OpenShift Monitoring

Red Hat OpenShift Monitoring provides monitoring for core platform components and user projects. The main OpenShift operator for monitoring is the *cluster monitoring operator*. This operator manages the deployment for the default monitoring stack.

The default OpenShift monitoring stack contains the following three components:

- *Prometheus*: Implementation of a systems monitoring and alerting toolkit that collects and stores metrics as time series data. The cluster monitoring operator relies on the Prometheus operator provided by Red Hat.

- *Thanos*: Implementation of different components that collaborate with Prometheus for aggregating data from other components, managing the retention of the metrics data, evaluating alerting rules, or exposing an API for querying metrics.
- *Alertmanager*: handles alerts sent by the Prometheus server, and sends alerts to notification systems.

This default monitoring stack provides observability for the OpenShift core platform components. When an OpenShift administrator activates monitoring for user workloads, the cluster monitoring operator creates a separate monitoring stack with Prometheus, Thanos, and Alertmanager instances.

There is another option: using the *cluster observability operator*. The cluster observability operator is an optional OpenShift operator that enables administrators to create standalone monitoring stacks, each independently configurable for use by different services and users. For example, if you need an independent Grafana console to create custom dashboards based on metrics, then you must use the cluster observability operator. The cluster observability operator provides the same Kubernetes custom resources, but in a different API. For example, there are two different API version definitions of the ServiceMonitor or PodMonitor resources:

- The `monitoring.coreos.com/v1`: from *cluster monitoring operator*
- The `monitoring.rhobs/v1alpha1`: from *cluster observability operator*

NOTE

This training content focuses on the default monitoring stack and its user workload monitoring stack. For more information about using the cluster observability operator, refer to the references section.

OpenShift Service Mesh uses the OpenShift monitoring stack to visualize the mesh topology, show traffic flow, and report on the health of your services by using the metrics from your applications running in your namespaces.

Distributed Tracing

Distributed tracing lets you follow a single request from its entry point into the system through all the services it uses. When handling a request, each service adds contextual information to the trace. Distributed tracing systems call this information a *span*. The collection of spans for a single request forms the complete trace.

Distributed tracing uses two components:

- The *OpenTelemetry project* collects observability data in distributed systems. OpenTelemetry provides vendor-neutral APIs for instrumentation, enabling instrumentation on the code, and sending data to the back-ends.
- The *Grafana Tempo project* acts as the tracing back-end. Grafana Tempo replaces Jaeger in the latest versions of OpenShift Service Mesh.

You must instrument applications to enable distributed tracing, although the OpenShift Service Mesh data plane can generate initial trace spans for services that are not fully instrumented. Proxies forward tracing information to collectors, and Kiali shows trace data with links to the tracing web UI.

Integrating OpenShift Service Mesh Projects in Default OpenShift Observability

To add projects or namespaces to the mesh for monitoring, OpenShift administrators must perform two actions: activate user workload monitoring and add the required label to the projects.

Activate user workload monitoring on the OpenShift cluster

When an OpenShift administrator activates the user workload monitoring, the cluster monitoring operator creates another monitoring stack. This stack is in a namespace distinct from the `openshift-monitoring` namespace and comprises new Prometheus, Thanos, and Alertmanager instances. You can use this monitoring stack to create the monitors for your services.

To activate user workload monitoring, the OpenShift administrator must modify or create a `cluster-monitoring-config` configuration map with the following content:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

When the administrator enables this parameter, the new monitoring stack runs in the `openshift-user-workload-monitoring` namespace.

Add the `istio-injection=enabled` label to the project that contains your services

This label causes Istio to create sidecar containers with Envoy proxies for each service in the namespace.

You can add the `istio-injection=enabled` label to a namespace by running this command with a cluster-admin user:

```
[user@host ~]$ oc label namespace \
namespace_name istio-injection=enabled
```

These operations enable the Prometheus instance used for user workload monitoring to scrape metrics from both your services and the injected Envoy proxy sidecars.

Kiali and the OpenShift Service Mesh Console Plug-in

Kiali centralizes all observability services and tools for OpenShift Service Mesh and visualizes the mesh. Kiali bases its implementation on the open-source Kiali community project. Red Hat provides a Kiali operator that deploys a Kiali server and the Kiali console. The OpenShift Service Mesh console plug-in (OSSMC) integrates all Kiali views into the OpenShift web console.

Key Capabilities

Kiali offers essential features for service mesh management:

Service Topology Visualization

Kiali generates live, interactive graphs that show how your microservices communicate with each other. These graphs show the following information:

- Service relationships and dependencies
- Traffic flow patterns and volumes
- Virtual services, destination rules, and gateways
- Circuit breakers and their status
- Request rates and latency metrics

Configuration Management

View and manage service mesh configurations, including:

- `VirtualService` and `DestinationRule` resources
- Gateway configurations
- Security policies
- Traffic routing rules

Observability and Monitoring

Monitor your service mesh health through:

- Real-time metrics and performance data
- Distributed tracing integration
- Health status calculations
- Problem detection and alerts

How OpenShift Service Mesh Observability Works

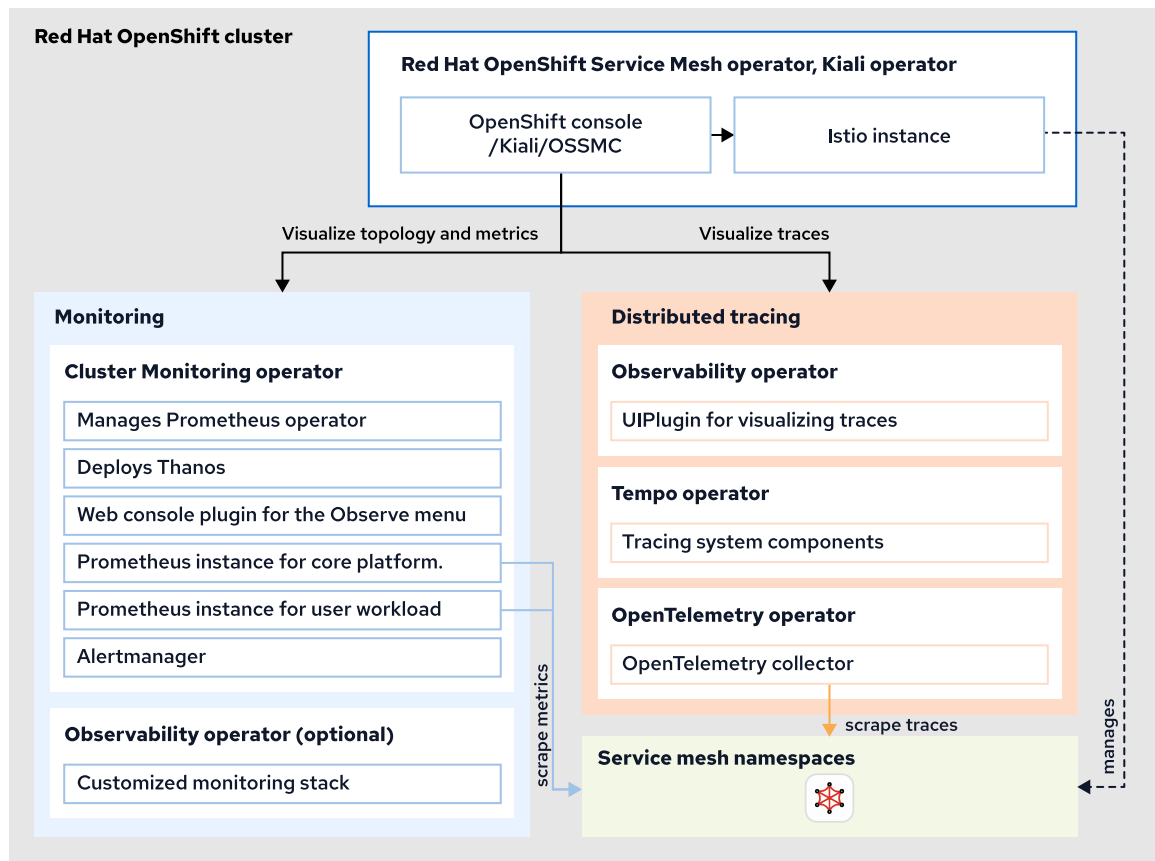


Figure 1.1: Integration of Kiali and OSSMC with OpenShift observability components

Kiali operates via the following data collection and processing pipeline:

1. *Data collection:* Prometheus scrapes metrics from your Envoy sidecar and application containers.
2. *Configuration retrieval:* Kiali queries the OpenShift and Istio APIs to fetch service mesh configurations.
3. *Data processing:* Kiali server processes and correlates metrics with configuration data.
4. *Visualization:* OSSMC presents the processed data through the OpenShift web console interface.

IMPORTANT

Prometheus integration is essential for Kiali functionality. Most Kiali features depend on Prometheus metrics data.

For more information about installing and using the observability operators, refer to the references section.

The Kiali Custom Resource

The Kiali operator provided by Red Hat includes the `kiali.io/v1alpha1.Kiali` custom resource. An instance of this custom resource runs in the same project or namespace as the Istio custom resource from the OpenShift Service Mesh operator. Thus, if you are using more than one mesh on the same OpenShift cluster in a multiple control plane topology, then you can have one Kiali console for each mesh. If you are using one mesh spanning more than one cluster, then you can configure Kiali to visualize it from a single point of view. For more information about using Kiali with multi cluster or multi control plane topologies, refer to the references section.

The following snippet shows a Kiali custom resource:

```
apiVersion: kiali.io/v1alpha1
kind: Kiali
metadata:
  name: kiali
spec:
  istio_namespace: istio-system ❶
  auth:
    strategy: openshift ❷
  deployment:
    ...output omitted...
  external_services: ❸
    grafana:
    ...output omitted...
    prometheus:
    ...output omitted...
    tracing:
    ...output omitted...
```

- ❶ Project where you install Kiali. In the same place that the Istio custom resource is running.
- ❷ Integrates the Kiali authentication with OpenShift Authentication.
- ❸ Entries for configuring all the components that Kiali needs.

Refer to the references section for the full reference of the `Kiali` custom resource.

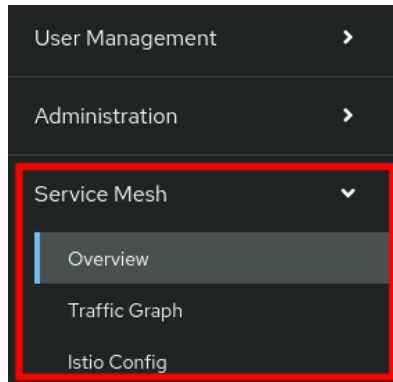
To obtain the web console URL for a running Kiali custom resource, run the following command:

```
[user@host ~]$ echo "https://$(oc get routes -n project_for_istio_and_kiali \
kiali -o jsonpath='{.spec.host}')
```

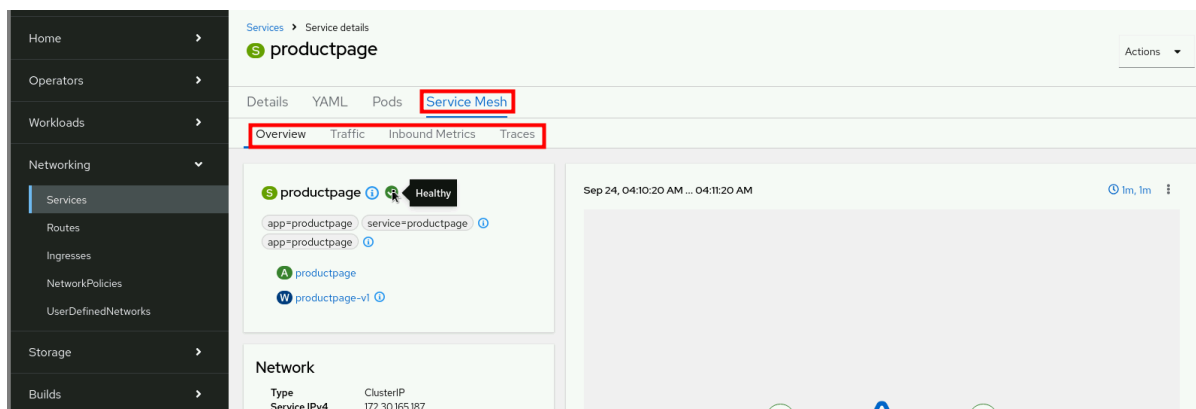
Kiali Operator and OpenShift Service Mesh Console

On OpenShift, the Kiali operator deploys a full Kiali console for each `Kiali` custom resource. If you have more than one mesh on the same OpenShift cluster, then you can run multiple Kiali consoles.

The operator also provides the OSSMC plug-in. When you activate this plug-in, a new **Service Mesh** menu entry shows in the OpenShift console's administrator view:



Other OpenShift and Kubernetes objects, such as Deployment and Service, show a new **Service Mesh** tab entry:



All Kiali console features are also in the OSSMC's additions to the OpenShift console.

IMPORTANT

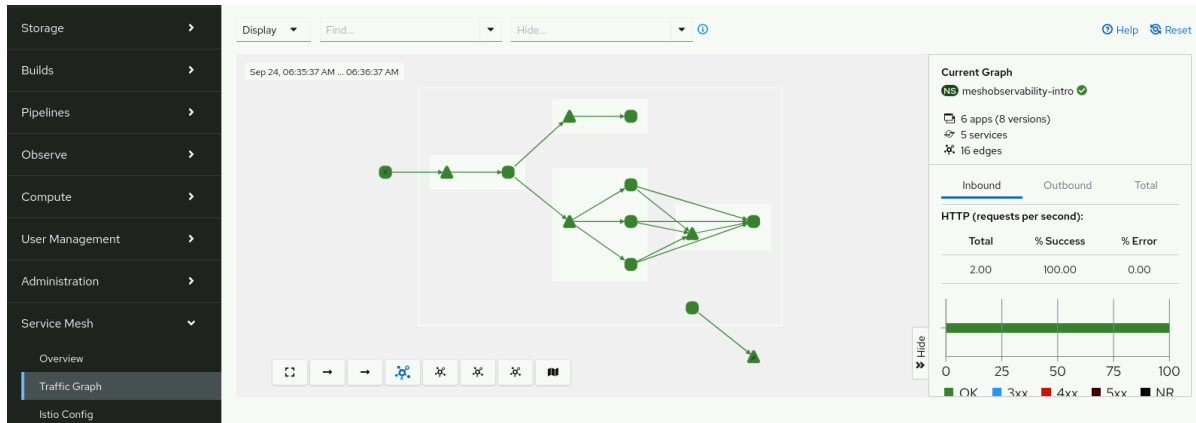
The only limitation of the OSSMC compared to the external Kiali console is that OSSMC can visualize only one mesh at a time. See the *Does Kiali support multiple meshes on a single cluster?* Knowledge Base content in the references section for more details. This training content focuses on the OSSMC visualization within the OpenShift web console.

The Kiali Traffic Graph

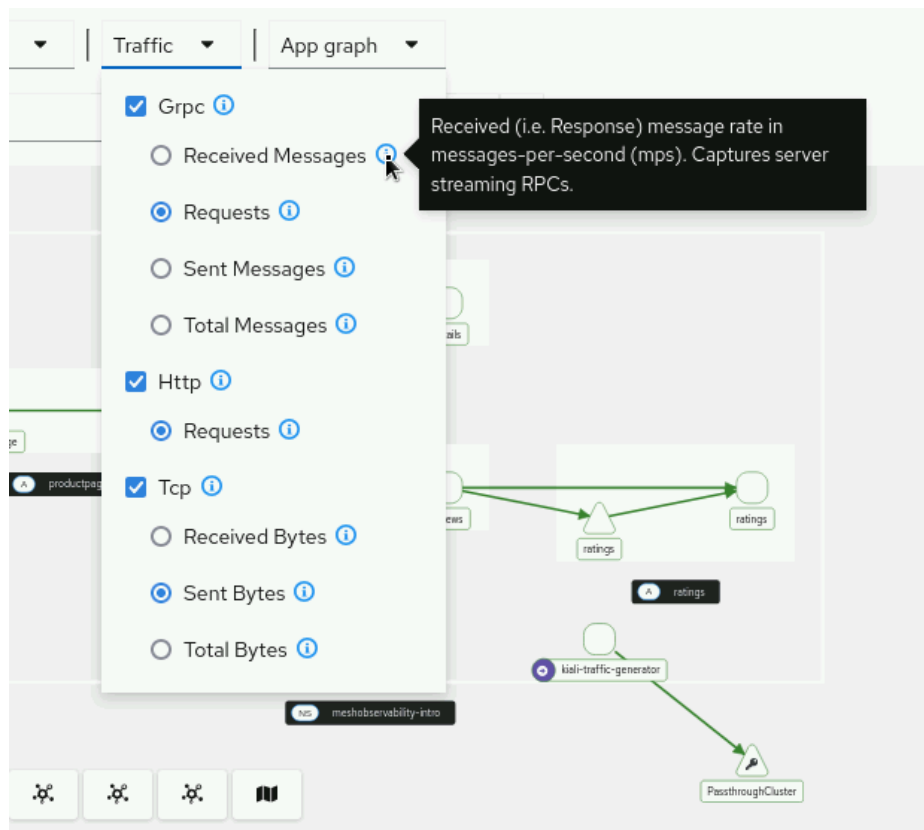
The OSSMC provides a topology *traffic graph* that visualizes live mesh traffic and correlates it with your Istio configuration. It enables you to do the following tasks:

- Assess service health
- Discover the routing behavior of virtual services or service entries
- Discover issues such as mTLS gaps, latency, and error spikes You can tailor the view with namespace and time-window selections, rich filters, and multiple visualization options that focus on different aspects of your mesh.

Access the Traffic Graph from the **Service Mesh** → **Traffic Graph** menu entry.



Some traffic graph elements show an exclamation mark icon. You can click the exclamation marks to get detailed information about the field near the icon.



The graph also provides the legend for the depicted nodes and edges.

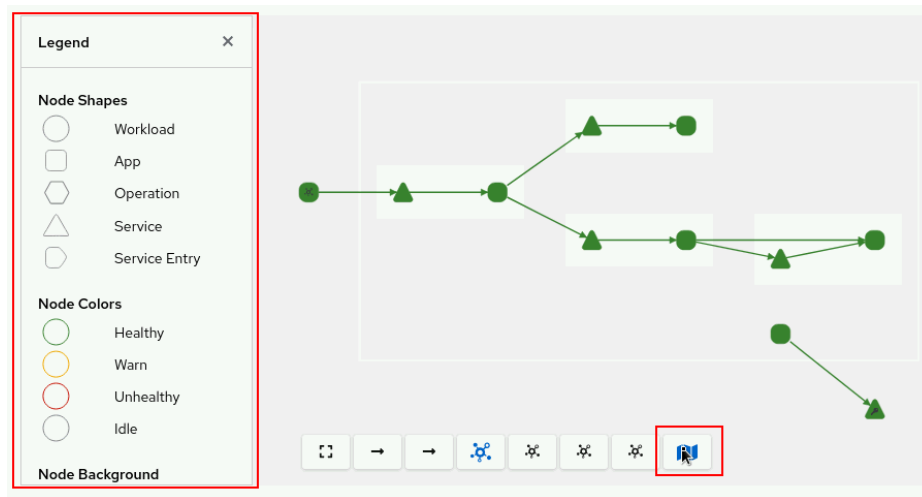


Figure 1.6: Graph legend

Graph types

Kiali offers graph renderings to match your level of detail, such as the following:

- **App graph:** Aggregates workloads by application labels for a logical view.
- **Service graph:** High-level view that aggregates traffic by Kubernetes services.
- **Versioned app graph:** Groups by application and breaks out versions to show version-specific traffic.
- **Workload graph:** Detailed, workload-to-workload communications.

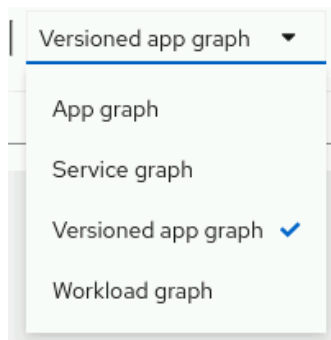


Figure 1.7: Types of traffic graphs.

Health and refresh features visualization

The color of nodes and edges indicates the health of your services. The red or orange colors indicate degraded nodes or unhealthy traffic between components. The health and telemetry data refresh automatically. You can pause the view to examine a moment in time, or even replay a selected period to re-analyze behavior.

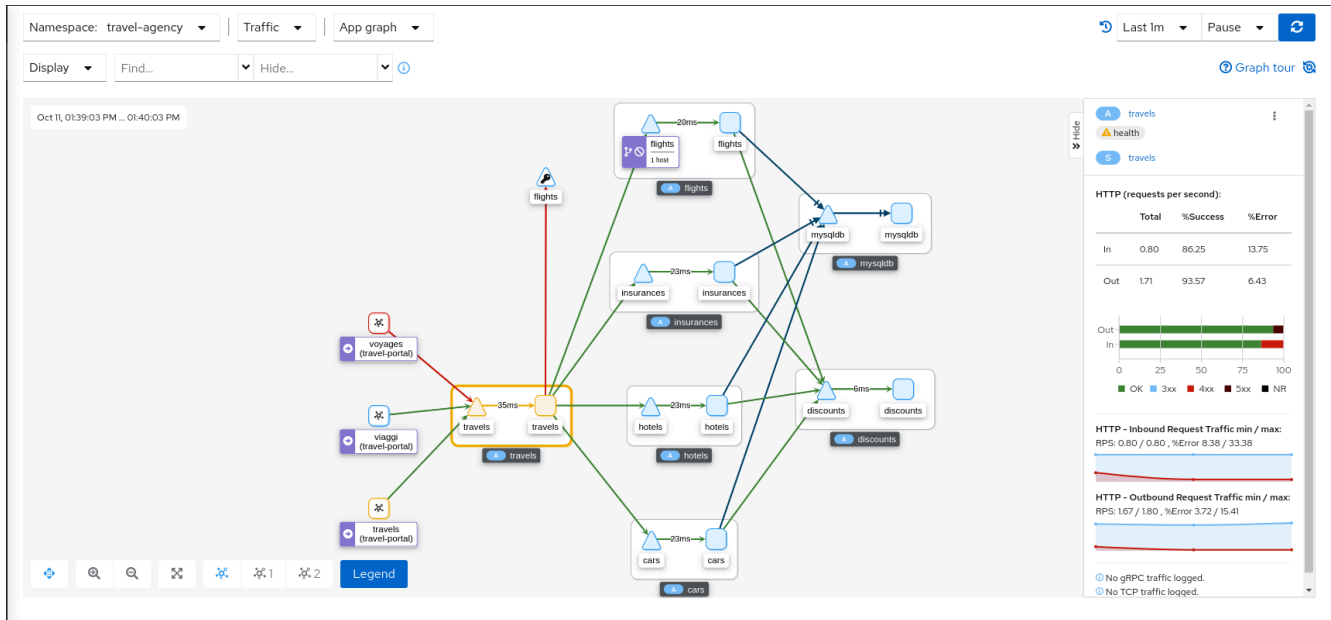


Figure 1.8: Different colours point to the health of services and communications

Node detail drill-down

Single-click selects a node in the graphic; double-click drills down to a node detail graph showing traffic strictly from that node's proxy perspective. You can go back to the main graph or drill into other nodes from there.

Side panel details

Selecting a node, edge, or boxed area opens a collapsible side panel that summarizes the selection and links to deeper information. This panel typically includes:

- Charts for request rates, error rates, and response times
- Health details and status
- Links to detailed mesh resources
- Response codes and host breakdowns
- Traces related to the selected component, if tracing is configured

You can hide the side panel by clicking on its **Hide** button.

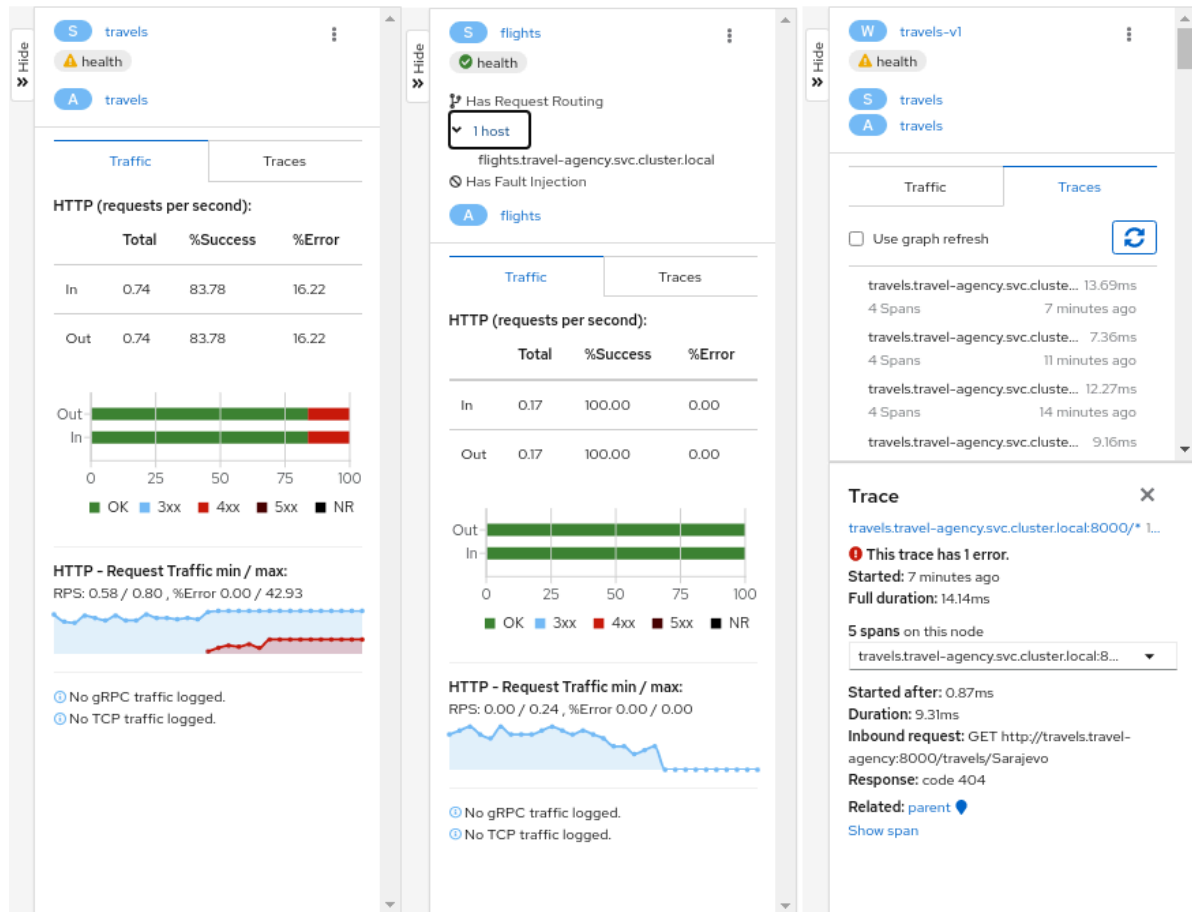


Figure 1.9: Examples of side panels showing information

Menu Display

Use the Display menu to toggle visual overlays and labels without changing the underlying traffic. Typical options include:

- Traffic animation and edge labels: request rate, error rate, response time
- Security indicators such as mTLS lock, missing sidecar, and health badges
- Istio configuration overlays: virtual services, destination rules, gateways, service entries
- Resilience markers: circuit breakers and fault injection
- Node visibility: show service nodes, application nodes, operation nodes
- Additional context: idle nodes or edges, protocol badges such as HTTP, gRPC, or TCP

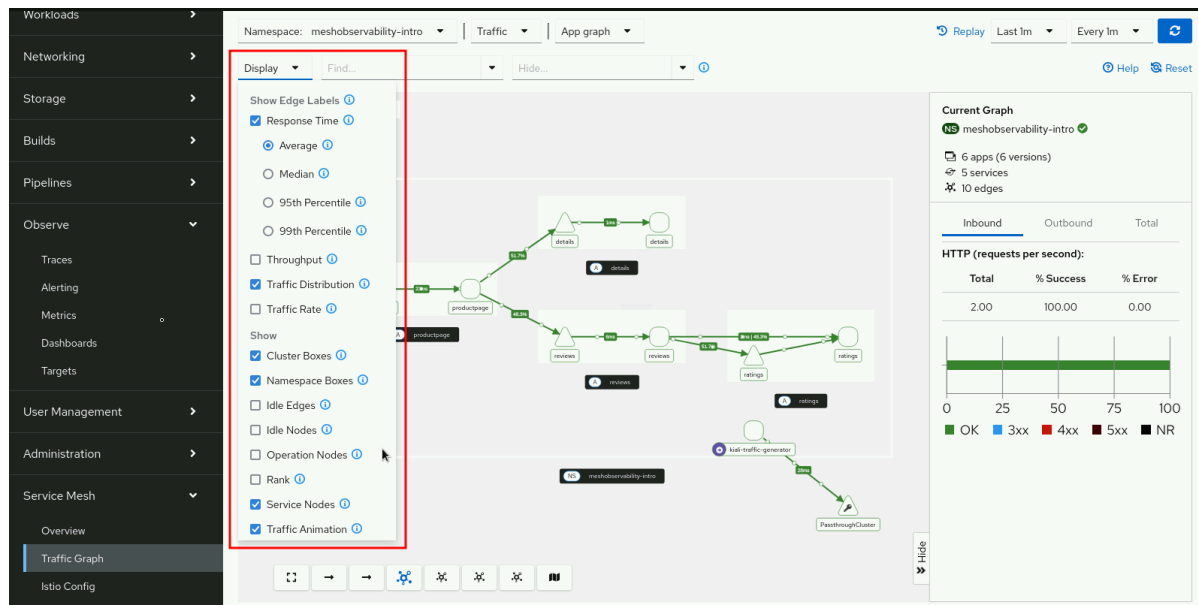


Figure 1.10: Some display menu options

Traffic animation

When you activate the traffic animation, the traffic graph shows ongoing communication between the graph nodes with animated figures. For HTTP traffic, the animated circles represent successful requests and the red diamonds represent errors. The density of the animated figures correlates with the request volume, and the animation speed correlates with response time. TCP traffic appears as offset circles whose speed reflects throughput.

Replay

Replay lets you examine historical traffic for a chosen time window. The graph state, including replay, is bookmarkable so that you can share a precise view with collaborators.



Figure 1.11: Replay page

REFERENCES

For more information about OpenShift Monitoring, refer to the *Monitoring* documentation at

https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index
(https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index)_{2/9}

For more information about Red Hat build of OpenTelemetry, refer to the *Red Hat build of OpenTelemetry* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/red_hat_build_of_opentelemetry/index (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/red_hat_build_of_opentelemetry/index)

For more information about Distributed Tracing, refer to the *Distributed Tracing* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index)

For more information about the cluster observability operator, refer to its documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/cluster_observability_operator/index (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/cluster_observability_operator/index)

For more information about the Kiali operator, refer to the *Kiali Operator provided by Red Hat* chapter of the OpenShift Service Mesh documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/observability/index#kiali-operator-provided-by-red-hat (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/observability/index#kiali-operator-provided-by-red-hat)

For more information about using Kiali in a multi cluster topology, refer to the *Installing Kiali in a multi-cluster mesh* section, in the *multi cluster topologies* chapter in the *Installing* guide from the OpenShift Service Mesh documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-installing-kiali-multi-cluster-mesh_ossm-multi-cluster-topologies (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-installing-kiali-multi-cluster-mesh_ossm-multi-cluster-topologies)

Kiali Custom Resource Reference (<https://v2-11.kiali.io/docs/configuration/kialis.kiali.io>)

Does Kiali support multiple meshes on a single cluster?
(<https://access.redhat.com/solutions/7128051>)



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책

(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침

(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정